

deepCausal: Multi-Environment Causal Training with Discrepancy Regularization

Project Report

October 12, 2025

Abstract

This report documents the methodology and experimental results for the **deepCausal** package, focusing on the usage script **Experiments/UsageR.R**. The package provides a training routine that jointly minimizes least-squares (LS) error across environments and a causal discrepancy (CD) term that penalizes differences in environment-specific errors. We describe the objective, the learning procedure (linear and neural network variants), the simulation systems used for benchmarking, and present quantitative results. We also highlight an important modeling caveat in System 2 and recommend a fix.

1 Overview

The goal is to learn predictive models that transfer across environments with distributional shifts. Given two environments (E_1, E_2), we train parameters β to minimize a convex combination of the pooled least-squares error and the absolute difference of mean squared errors across environments (the causal discrepancy):

$$\mathcal{L}(\beta) = (1 - \lambda) f_{\text{LS}}(\beta) + \lambda f_{\text{CD}}(\beta), \quad \lambda \in [0, 1]. \quad (1)$$

Here, f_{LS} is a sample-size weighted average of per-environment MSEs; f_{CD} is the absolute difference between the two environment MSEs.

2 Objective Function

For environments E_1 and E_2 with sample sizes n_1, n_2 , and mean squared errors $\text{MSE}(E_j)$:

$$f_{\text{LS}}(\beta) = \frac{n_1 \text{MSE}(E_1) + n_2 \text{MSE}(E_2)}{n_1 + n_2}, \quad (2)$$

$$f_{\text{CD}}(\beta) = |\text{MSE}(E_1) - \text{MSE}(E_2)|. \quad (3)$$

The implementation is provided by `obj_func()` in **R/functions.R**. Predictions are produced by a model function (linear or neural network). The objective is minimized using `optim()`.

3 Model Classes

3.1 Linear Model

When `hidden_sizes` is `NULL` or empty, `train_causal()` uses a linear predictor over features X with an intercept. The number of parameters equals the number of features plus one.

3.2 Feed-Forward Neural Network

If `hidden_sizes` is provided (e.g., `c(3,3)`), a feed-forward network with sigmoid activations in hidden layers and a linear output is used. The parameter count is determined by the layer sizes and includes biases. Both variants share the same objective.

4 Training and Evaluation

`train_causal()` accepts two environment datasets `data_G1`, `data_G2`, the tradeoff parameter λ , the `target` column name, and optional `cv_folds`. If `cv_folds > 1`, K-fold CV is run in each environment separately, models are fit on training folds, and performance is measured by RMSE on the pooled validation folds.

`evaluate_causal_model()` computes MSE and RMSE on a held-out test set using the trained parameters and the chosen predictor function.

5 Simulation Systems

Defined in `R/systems.R` via `pilot_systems()`:

- **System 1:** “Modified Non-linear System with External Actions”. Structural equations:

$$X_1 = \varepsilon_1 + A_1, \quad (4)$$

$$Y = \sin(X_1) + \varepsilon_Y, \quad (5)$$

$$X_2 = Y^2 + A_2 + \varepsilon_2. \quad (6)$$

Environments modulate the exogenous actions through $A_1 \sim \mathcal{N}(\mu_{A1}, \sigma_{A1}^2)$ and $A_2 \sim \mathcal{N}(\mu_{A2}, \sigma_{A2}^2)$.

- **System 2:** “7-Variable Medium System” with variables $X_1, \dots, X_7, Z$. Key relations:

$$X_1 = \varepsilon_1, \quad X_2 = X_1 + \varepsilon_2, \quad X_3 = X_1 + X_2 + \varepsilon_3, \quad (7)$$

$$Z = \sin(5X_2) + X_3^3 + \varepsilon_Z, \quad X_4 = X_2 + \varepsilon_4, \quad (8)$$

$$X_5 = Z + \varepsilon_5, \quad X_6 = Z + \varepsilon_6, \quad X_7 = X_6 + \varepsilon_7. \quad (9)$$

The generator returns a data frame with all variables and additionally sets $Y := X_5$. Environments are controlled via a single noise scale parameter `sd_eps`. To avoid target leakage in experiments, we drop column `X5` before training/evaluation.

- **System 3:** “Nonlinear Confounded Mediator System”:

$$X_1 = \varepsilon_1, \quad Z = \varepsilon_Z, \quad (10)$$

$$X_2 = \tanh(X_1) + 0.5Z + A + \varepsilon_2, \quad (11)$$

$$Y = 2\sin(X_2) + 0.7Z + \varepsilon_Y. \quad (12)$$

Environment controls: `mu_A`, `sd_A`, and common noise `sd_eps`.

- **System 4:** “Heteroskedastic Nonlinear Interaction System”:

$$X_1 = A + \varepsilon_1, \quad X_2 = \varepsilon_2, \quad (13)$$

$$Y = X_1X_2 + 0.5X_1^2 - 0.3X_2^2 + \varepsilon_Y, \quad \varepsilon_Y \sim \mathcal{N}(0, \sigma_Y^2(\text{env})). \quad (14)$$

Environment controls: `mu_A`, `sd_A`, feature noise `sd_eps`, and output noise `sdY`.

Table 1: Summary of cross-validated and test performance across systems.

System	Model	CV RMSE	Test RMSE	Test MSE
System 1	Linear	1.1066	1.7201	2.9587
System 1	NN (3,3)	1.1284	1.3599	1.8493
System 2	Linear	1.2860	1.1240	1.2633
System 2	NN (3,3)	102.5001	54.8840	3012.2540
System 3	Linear	1.6267	1.4908	2.2226
System 3	NN (3,3)	1.2741	0.9502	0.9029
System 4	Linear	2.7598	2.4687	6.0947
System 4	NN (3,3)	1.9501	1.6869	2.8458

6 Experimental Protocol

We run `Experiments/UsageR.R` which:

1. Generates training data for two environments per system using the respective environment parameters.
2. Trains a linear model and a neural network model with λ in $[0, 1]$ (here, $\lambda = 0.3$ or 0.6 as set in the script).
3. Performs 5-fold cross-validation and reports mean RMSE across folds.
4. Generates a held-out test set for each system and reports MSE/RMSE.

7 Results

All results below come from the recorded run of `UsageR.R`.

7.1 System 1

Training on two environments with different external action distributions.

- **Linear model** ($\lambda = 0.3$, CV=5): **CV RMSE** ≈ 1.1066 . Test **RMSE** ≈ 1.7201 , **MSE** ≈ 2.9587 .
- **Neural net** (hidden layers 3,3, $\lambda = 0.6$, CV=5): **CV RMSE** ≈ 1.1284 . Test **RMSE** ≈ 1.3599 , **MSE** ≈ 1.8493 .

7.2 System 2 (post-fix: no leakage)

Environments differ in the noise scale `sd_eps`. We explicitly drop `X5` (which equals `Y`) from predictors.

- **Linear model** ($\lambda = 0.3$, CV=5): **CV RMSE** ≈ 1.286 . Test **RMSE** ≈ 1.124 , **MSE** ≈ 1.263 .
- **Neural net** (hidden layers 3,3, $\lambda = 0.3$, CV=5): **CV RMSE** ≈ 102.50 . Test **RMSE** ≈ 54.88 , **MSE** ≈ 3012.25 .

7.3 System 3

- **Linear model** ($\lambda = 0.3$, CV=5): **CV RMSE** ≈ 1.6267 . **Test RMSE** ≈ 1.491 , **MSE** ≈ 2.2226 .
- **Neural net** (hidden layers 3,3, $\lambda = 0.3$, CV=5): **CV RMSE** ≈ 1.2741 . **Test RMSE** ≈ 0.9502 , **MSE** ≈ 0.9029 .

7.4 System 4

- **Linear model** ($\lambda = 0.3$, CV=5): **CV RMSE** ≈ 2.7598 . **Test RMSE** ≈ 2.4687 , **MSE** ≈ 6.0947 .
- **Neural net** (hidden layers 3,3, $\lambda = 0.3$, CV=5): **CV RMSE** ≈ 1.9501 . **Test RMSE** ≈ 1.6869 , **MSE** ≈ 2.8458 .

8 Recommendations

- **Avoid leakage:** For System 2 (and similar), ensure target-derived features are excluded from predictors.
- **Neural network stability:** Consider alternative initializations, learning-rate schedules (using an optimizer with gradients), or different architectures; the current `optim()`-based fitting can struggle on highly nonlinear tasks (Systems 2 and 4).
- **Hyperparameter tuning:** Explore λ , hidden layer sizes, and optimization settings.
- **Reproducibility:** Keep seeds and session info (we use `set.seed(123)` in the script).

9 Reproducibility and Code

Core files:

- R/functions.R: `obj_func()`, `train_causal()`, `evaluate_causal_model()`.
- R/systems.R: `pilot_systems()` with Systems 1–4 generators.
- Experiments/UsageR.R: experiment script orchestrating data generation, training, CV, and evaluation.

Minimal snippet of the usage script:

Listing 1: Excerpt from Experiments/UsageR.R

```
1 # System 2 environments (fixed)
2 data_env1_sys2 <- selected_system2$data_func(
3   n = 1000,
4   environment = list(sd_eps = 1)
5 )
6
7 data_env2_sys2 <- selected_system2$data_func(
8   n = 1000,
9   environment = list(sd_eps = 1.5)
10 )
11
12 model_linear_sys2 <- train_causal(
13   data_G1 = data_env1_sys2,
```

```
14 | data_G2 = data_env2_sys2,  
15 | lambda = 0.3,  
16 | target = "Y",  
17 | cv_folds = 5,  
18 | verbose = TRUE  
19 | )
```

10 Conclusion

The **deepCausal** framework combines pooled LS error with an environment discrepancy penalty to encourage stability across environments. System 1 shows reasonable performance for both linear and neural models. After fixing leakage, System 2 yields realistic errors for the linear model, while the neural model remains challenging under the present optimization. Systems 3 and 4 illustrate increasingly nonlinear and heteroskedastic dynamics where neural models can outperform linear baselines when training succeeds. Further work includes gradient-based training for neural networks and systematic hyperparameter tuning.